VIT Bhopal

Vityarthi E-Learning Platform

CSA 2001- Fundamentals in AI and ML

Project Report

Project : Smart Agri Advisor

Name : Anshu Sharma

Registration No.: 25BAI11353

School : SCAI

Program : CSE(AIML)

Professor : Dr. Pradeep Kumar Mishra

Smart Agri Advisor: AI-Powered Agricultural Decision Support System

1. The Problem I Chose

1.1 Observation from My Community

During my visits to my village in Uttar Pradesh, I spent time talking with farmers about their daily challenges. One conversation that stayed with me was with my uncle, who has been farming for over thirty years. He showed me a soil test report he had received from the agricultural extension office. The report contained numbers for nitrogen, phosphorus, potassium, pH, and other parameters. He looked at it with frustration and said, "I know these numbers are important, but I don't understand what they mean for my farm. I still end up planting what my neighbour plants or what I planted last year."

This was not an isolated incident. Over the next few weeks, I spoke with more than fifteen farmers in my community. Almost all of them had similar stories. They had soil test reports but did not know how to use them. They made cropping decisions based on tradition, advice from other farmers, or simply guesswork. One farmer told me he had planted cotton two years in a row on soil that was not suitable, losing over ₹2 lakh per hectare before switching to a different crop.

1.2 The Gap I Identified

What I observed was a clear gap between data and decision-making. Soil testing is becoming more common, with government programs providing subsidised or free tests. However, the reports are technical and assume a level of expertise that most farmers do not have. The farmers I spoke with wanted to use the data but did not know how.

At the same time, the advice available to farmers is often generic. Recommendations from fertiliser shops, neighbouring farmers, or even agricultural officers may not account for the specific conditions of a particular farm. Each field has its own soil chemistry, microclimate, and history. A recommendation that works for one farm may fail for another.

1.3 The Core Problem

This led me to define the core problem: Farmers lack an accessible, personalised tool that can take their specific soil and environmental data and translate it into clear, actionable recommendations about what to plant, how much to expect, and whether they will make a profit. Without such a tool, they continue to rely on guesswork, risking financial losses and inefficient use of resources.

I decided to take on this problem for my BYOP project. I wanted to build something that could bridge the gap between agricultural data and farmer decisions using the machine learning concepts I was learning in my course.

2. Why It Matters

2.1 Economic Impact

The economic consequences of poor cropping decisions are severe. According to the farmers I spoke with, a wrong crop choice can cause losses of ₹1 to 2 lakh per hectare. For a small farmer with two or three hectares, this can mean losing ₹3 to 6 lakh in a single season—enough to push a family into debt. Many farmers take loans for seeds, fertilisers, and labor. If the crop fails or underperforms, repaying those loans becomes impossible.

Beyond individual losses, inefficient farming affects the broader economy. India's agricultural sector employs nearly half of the country's workforce. When farmers struggle, rural economies suffer. Food prices become volatile. Resources like water and fertiliser are wasted when applied to unsuitable crops.

2.2 Human Impact

Beyond the numbers, there is a human cost. Farming is already one of the most unpredictable and stressful occupations. Farmers face uncertainty from weather, pests, market prices, and now climate change. Adding to this uncertainty the risk of making the wrong cropping decision creates constant anxiety.

During my conversations, I saw how this stress affects families. One farmer told me that every season, he lies awake at night worrying about whether he made the right choice. His wife described how the stress affects

their entire household. A tool that reduces this uncertainty, even a little, can improve quality of life in ways that are difficult to measure but very real.

2.3 Environmental Impact

Better decision-making also benefits the environment. When farmers apply fertiliser without knowing what their soil actually needs, they often over-apply. Excess nitrogen and phosphorus run off into water bodies, causing algal blooms and water pollution. When farmers plant water-intensive crops in areas with low rainfall, they deplete groundwater. By matching crops and inputs to actual soil and climate conditions, we can reduce waste and environmental damage.

2.4 Alignment with National Priorities

The Government of India has launched several initiatives to support farmers, including soil health cards, crop insurance schemes, and digital agriculture missions. However, these programs often stop at providing data or coverage. They do not necessarily help farmers interpret and act on that data. My project aligns with these national priorities by adding the missing layer of decision support.

3. My Approach to Solving It

3.1) Phase 1: Understanding the User

Before writing any code, I spent significant time understanding what farmers actually need. I conducted informal interviews with fifteen farmers in my village. I asked them about their current decision-making process, what information they had access to, and what kind of help they would find useful.

The findings were clear. First, farmers wanted to know which crop to plant based on their soil. Second, they wanted to know how much yield to expect. Third, they wanted to know whether they would make a profit. Fourth, they wanted to understand how changes in inputs like fertiliser or water would affect their outcome. These four needs became the pillars of my application.

3.2> Phase 2: Data Collection

I needed data to train my machine learning models. I started by collecting real soil test reports. Through family connections, I obtained reports from 50 farms in my district. Each report contained values for nitrogen, phosphorus, potassium, pH, and other parameters. I also collected yield data for those farms from the farmers themselves, along with information about the crops they had planted.

Fifty samples were not enough to train robust machine learning models. To address this, I generated synthetic data. For each real sample, I created

multiple synthetic variations by adding controlled random noise to each parameter. The noise was small enough to keep the values realistic but large enough to create variety. This gave me a dataset of 500 samples, which was sufficient for training.

I also collected market price data from local mandi records covering the years 2022 to 2025. This allowed me to calculate realistic revenue and profit figures.

3.3> Phase 3: Model Development

I experimented with several machine learning algorithms to find the best fit for my problem.

Decision Tree:I first tried a decision tree classifier for crop recommendation. It achieved around 72 percent accuracy but showed signs of overfitting—performing well on training data but poorly on new data.

Support Vector Machine: Next, I tried SVM. Accuracy improved to about 78 percent. However, SVM does not provide feature importance scores, which I wanted to show to users.

Random Forest: Finally, I tried Random Forest. It achieved 85 percent accuracy on the test set. It also provided feature importance scores that I could display to help users understand what factors matter most. Random Forest is an ensemble method that combines multiple decision trees, which reduces overfitting and improves generalisation.

I used Random Forest for both crop recommendation (classification) and yield prediction (regression). For yield prediction, I achieved an R^2 score of 0.85, meaning the input features explained 85 percent of the variation in yield.

3.4 Phase 4: Building the Application

I built the application using Python and Streamline. I chose Streamlit because it allowed me to focus on the machine learning logic without spending time on frontend development. Streamlit automatically creates a web interface from Python code, which was perfect for this project.

The application is structured as a single Python script. It loads the trained models, then creates input fields for land area, crop selection, soil parameters, and environmental conditions. When the user enters values, the application runs the models, generates predictions, calculates financial metrics, and displays the results along with interactive graphs.

3.5> Phase 5: Adding Visualisations

To help users understand the models and build trust, I added several visualizations using matplotlib.

Sensitivity Graphs: I created graphs showing how yield changes with rainfall, temperature, nitrogen, and pH. These graphs allow users to explore different scenarios and understand the non-linear relationships that the model has learned.

Feature Importance Chart: I created a bar chart showing which factors most influence yield. Nitrogen and rainfall consistently appeared as the most important features, which aligns with agricultural science.

Correlation Matrix: I included a correlation matrix showing relationships between all parameters. This helps users understand how different factors interact.

3.6 Phase 6: Testing and Refinement

I tested the application with three farmers who had agreed to try it. They entered data from their soil test reports and compared the recommendations with their own experience and with advice from agricultural officers.

The feedback was valuable. One farmer pointed out that some terminology was confusing. I simplified labels and added explanatory text. Another farmer suggested that the profit calculation should account for different market prices in different seasons. I updated the financial model to use seasonal price data from mandi records.

After several rounds of feedback and refinement, the application was ready. The farmers who tested it said they found it useful and would consider using it for future seasons.

4. Key Decisions I Made

Decision 1: Random Forest Instead of Deep Learning

I chose Random Forest over deep learning approaches because my dataset was relatively small. Deep learning models typically require thousands of samples to train effectively. With only 500 samples, deep learning would likely overfit or fail to generalize. Random Forest works well with smaller datasets and provides interpretable results through feature importance scores. This decision prioritized practicality over chasing the latest technology.

Decision 2: Including Financial Analysis

I could have stopped at crop recommendation and yield prediction, but I decided to add financial analysis because farmers care about profit, not just yield. A farmer might achieve high yield but still lose money if input costs are too high or market prices are low. By calculating revenue, total cost, net profit, and return on investment, the tool provides actionable financial insights that directly impact decision-making. This decision made the tool more valuable to users.

Decision 3: Interactive Sensitivity Graphs

I decided to include interactive graphs showing how yield changes with different inputs. This decision came from observing that farmers wanted to understand risk and explore what-if scenarios. The graphs allow them to see,

for example, what happens to yield if rainfall is lower than expected or if they reduce fertilizer use. This builds trust in the model and helps farmers make more informed decisions.

Decision 4: Streamlit as the Framework

I chose Streamlit because it allowed me to focus on Python and machine learning without spending time on frontend development. I considered using Flask or Django, but those would have required writing HTML, CSS, and JavaScript separately. Streamlit automatically creates a professional interface from Python code, which saved significant development time and allowed me to iterate quickly based on user feedback.

Decision 5: Including Future Climate Projections

I added a future outlook module to address climate change, which farmers increasingly worry about. This feature shows how yield might change with temperature increases based on climate model projections. It makes the tool forward-looking and demonstrates that machine learning can be used for scenario planning, not just current predictions.

Decision 6: Limiting to 7 Key Features

I limited the model to 7 key features: nitrogen, phosphorus, potassium, temperature, humidity, pH, and rainfall. These are the features most commonly measured in soil test reports and most relevant to crop growth. With limited data, using more features would have caused overfitting—the curse of dimensionality. This decision balanced comprehensiveness with model performance.

Decision 7: Combining Real and Synthetic Data

When I realized that 50 real samples were not enough for robust training, I decided to generate synthetic data. I could have used only the real data and accepted lower performance, but I chose to expand the dataset using controlled noise. This decision improved model accuracy while maintaining realistic patterns because the synthetic data was derived from real samples.

Decision 8: Simple, Clear Interface

I decided to prioritize simplicity in the user interface. I avoided technical jargon where possible and used plain language for labels and explanations. When technical terms like "feature importance" were necessary, I added brief explanations. This decision came from user testing, where farmers told me they were confused by complex terminology.

5. Challenges I Faced

Challenge 1: Data Scarcity

The most significant challenge was the lack of data. I needed hundreds of soil test reports to train a reliable machine learning model, but I only had access to 50 real reports. Many farmers were hesitant to share their reports, viewing them as private information. Some did not keep copies after using them. Others had reports that were several years old and may not reflect current soil conditions.

How I Addressed It: I generated synthetic data by adding controlled random noise to the real reports. For each real sample, I created multiple synthetic variations where each parameter was varied by a small random amount within realistic bounds. This gave me a dataset of 500 samples. I validated that the synthetic data maintained realistic patterns by checking that the ranges and correlations matched agricultural science.

Challenge 2: Model Overfitting

In early training runs, my models performed very well on training data but poorly on test data. For example, one model achieved 95 percent accuracy on training but only 70 percent on test. This was classic overfitting—the model was memorizing the training examples rather than learning general patterns.

How I Addressed It: I tuned the hyperparameters of the Random Forest model. I reduced the maximum depth from 20 to 8, which limits how specific the trees can become. I increased the minimum samples per split from 2 to 5, ensuring that splits are based on sufficient data. I also increased the minimum samples per leaf to prevent leaves from representing single data points. After these adjustments, the gap between training and test accuracy narrowed significantly.

Challenge 3: Feature Correlation

Some features in my dataset are naturally correlated. For example, farms with high nitrogen often also have high organic matter, which correlates with other nutrients. This multicollinearity can cause instability in some algorithms.

How I Addressed It: Random Forest handles correlated features well because each tree in the forest selects a random subset of features at each split. This reduces the impact of correlations. I did not need to remove or transform features, which preserved the interpretability of the model.

Challenge 4: User Interface Complexity

When I first showed the application to farmers, they found it overwhelming. There were too many input fields, and the terminology was confusing. One farmer asked, "What is humidity? I don't measure that." Another was confused by the profit calculation, wondering why the cost numbers were different from what he actually paid.

How I Addressed It: I simplified the interface. I added default values for parameters that farmers might not have, with an explanation that they could adjust them later. I changed labels to use simpler terms. I added tooltips and help text. For the profit calculation, I showed a breakdown of costs so farmers could see how the numbers were derived. I also tested the revised interface with the same farmers and incorporated their feedback.

Challenge 5: Learning Streamlit

I had never used Streamlit before this project. Learning a new framework while also building a complex application was challenging. I struggled with state management—how to keep values consistent across different parts of the application. I also had to learn how to integrate matplotlib graphs into the Streamlit interface.

How I Addressed It: I spent dedicated time going through the Streamlit documentation and tutorials. I started with a minimal version of the application and gradually added features. For state management, I learned to use Streamlit's session state feature. For graphs, I learned that Streamlit displays matplotlib figures directly. The learning curve was steep, but once I understood the patterns, development became much faster.

Challenge 6: Validating with Farmers

Getting feedback from farmers was essential but not always easy. Farmers are busy, especially during planting and harvesting seasons. Finding time to sit with them and go through the application required patience and flexibility.

How I Addressed It: I scheduled sessions during the off-season when farmers had more time. I visited them at their farms rather than asking them to come to me. I kept the sessions short and focused on specific features rather than trying to cover everything at once. I also made sure to listen to their feedback without being defensive, even when it was critical.

Challenge 7: Handling Seasonal Variation

Market prices for crops vary significantly by season. A fixed price assumption would lead to inaccurate profit calculations. Similarly, the relationship between rainfall and yield depends on when the rain falls, not just how much.

How I Addressed It: I incorporated seasonal price data from mandi records. The application uses the average price for the season when the crop is harvested. For rainfall, I acknowledge that the model uses total annual rainfall, which is a simplification. In future versions, I plan to incorporate monthly rainfall distribution.

6. What I Learned

1) Technical Learnings

Applying Machine Learning to Real Problems: This project taught me that machine learning is not just about running algorithms on clean datasets. Real problems come with messy, limited data. I learned how to work with what is available, how to generate synthetic data responsibly, and how to tune models for real-world constraints.

Importance of Hyperparameter Tuning: I learned that small changes in hyperparameters can have large effects on model performance. Reducing max_depth from 20 to 8 made the difference between an overfit model and one that generalized well. I now understand why hyperparameter tuning is a critical step in any machine learning project.

Building End-to-End Applications: I learned how to take a machine learning model from development to deployment. Using Streamlit, I built a complete interactive application that non-technical users can actually use. This skill—turning a model into a usable tool—is valuable in any data science role.

Data Collection and Synthesis: I learned practical techniques for collecting real-world data and augmenting it with synthetic samples. I also learned the importance of validating synthetic data to ensure it remains realistic.

Visualization for Communication: I learned that graphs and charts are not just for analysis; they are for communication. The sensitivity graphs and feature importance chart helped users understand the model and build trust in its recommendations.

Model Deployment with job lib

2.) Domain Learnings

Agricultural Science: I learned more about soil chemistry than I expected. I now understand what nitrogen, phosphorus, and potassium do for plants, why pH matters, and how different crops have different requirements. This knowledge helped me design better features and interpret the model outputs.

Farm Economics: I learned about the economics of farming, including the various costs that go into production. Fixed costs like land preparation and seeds. Variable costs like fertilizer, water, and labor. I learned how profit margins can vary dramatically with small changes in yield or price.

Farmer Decision-Making: I learned that farmers make decisions based on a complex mix of factors. Tradition, advice from other farmers, input costs, expected prices, risk tolerance, and even family obligations all play a role. A tool that provides recommendations must respect this complexity and offer clear, actionable advice without oversimplifying.

Climate Change Concerns: Through conversations with farmers, I learned how seriously they take climate change. Many described how rainfall

patterns have become less predictable, how summers are hotter, and how traditional knowledge no longer applies. This made me realize that any agricultural tool must account for changing conditions, not just historical patterns.

3.) Personal Learnings

User-Centered Design: I learned that building something useful requires understanding the user first. The features I thought would be most valuable were not always the ones farmers wanted. Early versions had too many technical details and not enough practical guidance. Listening to feedback and iterating based on what users actually need made the final product much better.

Simplicity is a Feature: I learned that simple, well-executed projects can have real impact. There is no need to use the most complex algorithms or the largest datasets. Doing something small well is better than doing something large poorly. This project is not large in scale, but it is thoughtful and grounded in real user needs.

Patience with Feedback: I learned to be patient with feedback, even when it was critical. When a farmer told me the interface was confusing, my first instinct was to defend it. But when I listened and made changes based on what they said, the application improved. I learned that feedback is not criticism of me; it is information that helps me build a better product.

Connecting Theory to Practice: This project helped me connect the machine learning concepts I was learning in class to a real-world problem. Classification, regression, overfitting, hyperparameters, feature importance—

these were not just abstract ideas anymore. They were tools I used to solve a problem that matters to real people.

The Value of Real Data: I learned that there is no substitute for real data. Synthetic data helped, but it was the real soil test reports from actual farms that gave the models credibility. When a farmer sees that the model was trained partly on data from farms like theirs, they trust it more.

Building for Impact: Finally, I learned that the best projects are those that can make a difference. This application may not change agriculture overnight, but it can help a few farmers make better decisions. That is enough. Knowing that my work could help someone in my community makes all the effort worthwhile.

Declaration

I, Anshu Sharma , 25BAI11353 ,hereby declare that this project titled " Smart Agri Advisor" is my original work. All code was written by me using Python and displayed using Streamlit. The project applies Artificial Intelligence and machine learning concepts learned during the course to solve a real-world agricultural problem.